Traffic Sign Classification with Convolutional Neural Network

Abstract

This project classifies traffic signs from the German Traffic Sign Recognition Benchmark (GTSRB) into 43 classes. The work includes data exploration, preprocessing, CNN architecture comparison, training strategy, hyperparameter tuning, and error analysis. The strongest model is a VGG-style CNN trained on normalized 32×32 RGB images with data augmentation, batch normalization, dropout, RMSprop and learning-rate decay. It achieved 98.74% test accuracy, macro F1 of 0.9794, and weighted F1 of 0.9882.

1 Problem Formulation

The task is supervised multi-class image classification. Each input is a cropped RGB traffic sign image and the target is one of 43 GTSRB classes. This problem is relevant for driver assistance and autonomous driving because signs encode speed limits, priority rules, warnings and restrictions that affect driving decisions [13]. The output is a single class label, so the model must learn the visual features that distinguish signs with similar colours, borders, and symbols. Some differences are large, for example warning signs versus speed-limit signs, but others are small, such as different speed-limit digits.

The formulation matters because it determines the correct model family. Models such as YOLO predict both bounding boxes and classes in full images [8], but given the fact that the dataset already has cropped images of only traffic signs, the task is not an object detection problem. A CNN classifier is therefore the correct method for this assignment, while a detector would only be needed for a different project using full road-scene images.

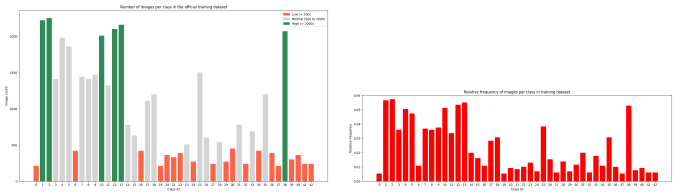
The project objective was not only to obtain a high accuracy, but also to build a reproducible deep learning experiment. For that reason the notebook records the dataset path, image size, split sizes, seed, training settings, model parameters, and final metrics. The report also uses class-wise metrics and error analysis because a traffic sign classifier can look strong in global accuracy while still failing on small or visually similar classes.

2 Data and Preprocessing

The dataset used is the official GTSRB files: 39,209 rows in `Train.csv`, 12,630 rows in `Test.csv`. The split was taken from the kaggle competition. The test set was kept for final evaluation, while the training set was split into training (80%) and validation (20%). This follows the standard train/validation/test pipeline: training data fits parameters, validation data guides model decisions and hyperparameters tuning and test data estimates final generalization.

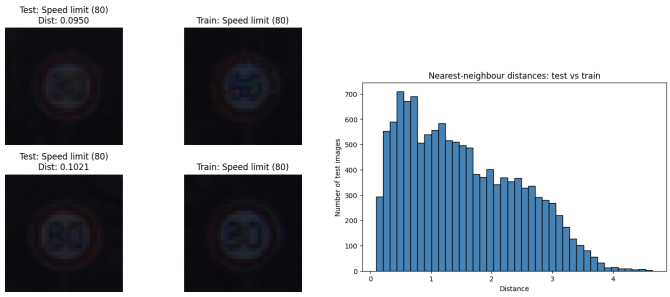
These datasets include resized images of 32×32 and kept as three-channel RGB tensors, which means all the images had the size of $32 \times 32 \times 3$. This resolution is small enough to train quickly on local hardware, while still keeping the colour, shape, and symbol information needed for GTSRB. Pixel values were divided by 255 so that the inputs were in $[0, 1]$, which improves numerical stability for gradient-based learning [2]. Labels were one-hot encoded because the network uses a softmax output and categorical cross-entropy. A fixed random seed (=1) was used for Python, NumPy and TensorFlow before model creation, making the comparisons more reproducible.

Exploration showed that the classes are imbalanced: some classes have only 210–450 training images, while several speed-limit and priority classes have more than 2,000. The imbalance is shown in Figures 1 in red being the underrepresented classes and in green are the overrepresented ones and in **??** was analysed their relative frequencies, some of the classes having less than 1% of the enitre data. This affects evaluation because frequent classes can dominate accuracy and lead to data leakage. For that reason the final comparison reports macro F1 and weighted F1, not only test accuracy. The maximum value of pixels is 255 and the minimum is 0 in both datasets this lead us to normalisation as presented earlier. Furthermore, brightness varies strongly within the images: the darkest image had mean brightness 6.64, while the brightest had 247.49. Many examples are affected by shadows, exposure, or low contrast, which can make the model task hard. Instead of removing these difficult examples that can be usually met during traffic, the training data used random brightness and contrast augmentation up to 15%, which is a standard way to improve robustness and reduce overfitting[9].



(a) Images per class. **(b)** Relative frequencies.
Figure 1: Class imbalance in the GTSRB training set.

The notebook also included data-quality checks, such as missing values and no duplicate groups inside the train and test, none of them were found. A hash-based leakage check found 5 train/test and 3 validation/test exact overlaps after preprocessing. These were removed from train/validation only, leaving the test set unchanged. This is important because even a small number of exact overlaps can make evaluation slightly too optimistic. After removal the final arrays were 31,362 training images, 7,839 validation images and 12,630 test images. A nearest-neighbour was used as a sanity check for near-duplicates, since train/test near-duplicates can bias image evaluation [1]. The distance distribution was plotted 2; after exact overlap removal. The cutoff distance < 0.1 was used only as an exploratory threshold from this distribution, not as an universal rule. It flagged one train/test pair, which was kept because it was not an exact duplicate and there was not enough evidence to treat it as confirmed leakage.



(a) Closest pairs. **(b)** NN distances.
Figure 2: Train/test similarity checks.

3 Model Design Choices

CNNs are suitable here because traffic signs are defined by local visual patterns such as borders, symbols, colours, digits, arrows and pictograms. Convolutional layers share parameters across spatial locations and build hierarchical features, which is the standard reason CNNs work well for image classification [5, 2]. Early layers can detect simple edges and colour transitions, while later layers can combine them into sign-level features such as a red circular border, a triangular warning shape, or an internal symbol. Before each model everything was reset for comparison

The first experiment included an AlexNet-style model adapted to 32×32 images, with five convolutional layers and two large fully connected layers of 4096 units each. It was compiled with the Adam optimizer, categorical cross-entropy loss and accuracy as the metric, then trained for 20 epochs with a batch size of 64 using the validation set for tuning. This baseline was useful because it showed that the task was learnable, but they were too large for 32×32 signs because most parameters were in dense layers rather than in reusable convolutional features. It had 37,4m (142.87 MB) parameters, most of them in the fully connected classifier. This increased computational cost and made the model prone to overfitting, because a large fully connected head can memorize training-specific patterns instead of relying on compact convolutional features.

The AlexNet-style baseline was also tested with data augmentation layer added before the convolutional blocks. This made the training examples more variable and slightly improved the final validation accuracy from 95.14% without augmentation to 96.16% with augmentation. However, both AlexNet-style runs had very large and unstable loss values, so augmentation helped accuracy but did not solve the main problem that the architecture was too large for 32×32 inputs.

Afterwards, `model1` used data augmentation with three convolutional blocks, each containing convolutional layer(s), max pooling, batch normalization and dropout layers, followed by a dense classifier with 4000, 4000, and 1000 hidden units. It was compiled similarly with the previous model for comparison reasons. After 20 epochs it reached 95.52% validation accuracy, but the loss values were large and unstable. This showed that the model could learn useful features, but its dense head made it unnecessarily large and not as reliable as the later VGG-style models.

Subsequently, a VGG-style arhitecture based on repeated 3×3 convolutions was implemented [10]. Small filters are appropriate here because the images are low-resolution and the important details are local. Stacking two 3×3 layers gives the network more non-linear transformations than one larger filter, while keeping the architecture simple to explain. Pooling gradually reduces the spatial size, so the network moves from local pixel patterns to more abstract sign representations. Batch normalization was added after pooling to stabilize the activations during training, while dropout was used after each block and before the final classifier to reduce overfitting.

The model progression was designed to test capacity rather than only chase accuracy:

- `Model 2`: stacked 3×3 convolution blocks with batch normalization, dropout, and a flatten-based dense head.
- `Model 2c`: replaced the heavy classifier with global average pooling, reducing parameters while keeping strong validation performance [6].
- `Model 2cl`: reduced the final convolution block from 256 to 128 filters, giving 605,643 parameters instead of 1,212,619 .
- `Model 2cll`: halved filters further, giving 160,507 parameters and testing the accuracy-efficiency trade-off.
- `Model 2cllh`: kept structure and added He initialization for the ReLU convolutional and dense layers and tested whether better initialization improved the model
- `Model 2cllh2`: kept the same architecture as `Model 2cllh`, but changed the optimizer to AdamW with weight decay; it reached 99.64% final validation accuracy.
- `Model 2cllh2l`: reused the same architecture, but tested a smaller AdamW learning rate and weight decay; it learned too slowly and only reached 15.51% final validation accuracy.
- `Model 2cllh2l_SGD`: Same architecture with SGD as compiler, momentum, and F1 metrics during training; it reached 99.49% final validation accuracy.
- `Model 2cl_reg`: reused the `Model 2cl` architecture with L2 regularization on

convolutional and dense layers; it reached 99.77% final validation accuracy.

- `Model 2cl_lrdecay`: reused the `Model 2cl` architecture with Adam and a ReduceLROnPlateau learning-rate scheduler; it reached 99.89% final validation accuracy.
- `Model 2cl RMS`: kept the `Model 2cl` architecture, but changed the optimizer to RMSprop and used ReduceLROnPlateau; it reached the best final validation accuracy of 99.95%.
- `Model 2cl RMS ResNet`: added residual skip connections to the `Model 2cl RMS` idea; it had 617,291 parameters and reached 99.55% final validation accuracy. Weaker accuracy and more computational expensive
- `Model 2cl RMS Inception (GoogLeNet)`: tested a multi-branch Inception-style version with RMSprop and learning-rate scheduling; it used only 104,283 parameters and reached 99.55% final validation accuracy. But it has big fluctuations in terms of validation accuracy

A fuller comparison, such as parameter counts, memory use, optimizers, final losses, and final/best validation accuracies is provided in Table 1.

Table 1: Summary of CNN architecture and optimizer experiments.

Metric	M2	M2c	M2cl	M2cll	H	H2	H2l	SGD	Reg	LR	RMS	Res	Inc
Change	Flat	GAP	128f	Half	He	AdamW	Low LR	SGD	L2	LR decay	RMS	Res.	Inc.
Params	1.72M	1.21M	605K	161K	161K	161K	161K	161K	605K	605K	605K	617K	104K
Trainable	1.72M	1.21M	605K	160K	160K	160K	160K	160K	605K	605K	605K	616K	104K
Memory	6.57M	4.63M	2.31M	627K	627K	627K	627K	627K	2.31M	2.31M	2.31M	2.35M	407K
Opt.	Adam	Adam	Adam	Adam	Adam	AdamW	AdamW	SGD	Adam	Adam	RMS	RMS	RMS
Train acc.	98.15	98.55	98.47	97.45	96.88	96.99	9.40	95.55	98.26	98.94	99.37	98.79	97.24
Val. acc.	99.63	99.64	99.81	99.63	99.45	99.64	15.51	99.49	99.77	99.89	99.95	99.55	99.55
Best val.	99.63	99.82	99.81	99.63	99.69	99.67	15.51	99.49	99.77	99.91	99.95	99.90	99.55
Train loss	.1595	.0720	.0761	.1069	.1180	.1198	3.6909	.1664	.2052	.0498	.0306	.0444	.1012
Val. loss	.0317	.0099	.0059	.0119	.0152	.0136	3.1999	.0189	.1274	.0047	.0024	.0175	.0143

The final `Model 2cl RMS` architecture has four convolutional stages, each followed by max pooling, batch normalization and dropout, then global average pooling, one 128-unit dense layer and a 43-class softmax output. Batch normalization stabilizes activations during training [4], while dropout reduces co-adaptation and overfitting [12]. Global average pooling keeps the classifier small by summarizing feature maps before the dense layer. This addresses the main weakness of the earlier models, where the classifier head was too large for 32×32 input images.

4 Training Strategy and Tuning

All models were trained for 20 epochs with a batch size of 64, using categorical cross-entropy because the task is single-label multi-class classification. The first VGG-style models used Adam as a strong adaptive baseline optimizer [2]. After the AlexNet-style and dense-head models proved inefficient, the tuning focused first on reducing capacity: replacing the flatten-based classifier with global average pooling, reducing the number of filters and testing a smaller lightweight version.

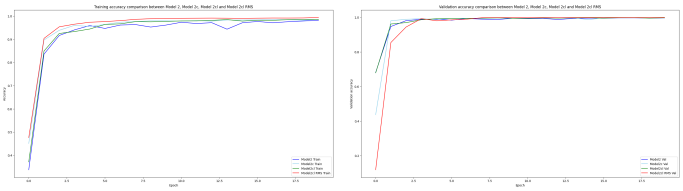
After the architecture had been simplified, the next step was to test whether training could be improved without making the model larger. For this reason, I compared He initialization, AdamW with weight decay, L2 regularization, SGD with momentum, RMSprop and ReduceLROnPlateau. Two functional exploratory variants were also tested: a ResNet-style model with residual skip connections [3] and an Inception-style model (GoogLeNet) with multi-branch convolutions [14]. The best result came from `Model 2cl RMS`, which kept the `Model 2cl` architecture but changed the optimizer to RMSprop with learning-rate decay [15].

Augmentation was applied only during training, so validation and test results were measured on unchanged images. This improved robustness during fitting while keeping evaluation realistic. Final model selection was checked using test accuracy and F1-scores, since validation accuracy alone can hide class-level weaknesses.

5 Performance Analysis

Table 2 shows that `Model 2cl RMS` achieved the best test accuracy (98.74%), macro F1 (0.9794), and weighted F1 (0.9882). Since GTSRB is imbalanced, macro F1 is the more important measure because it gives each class equal weight, so minority classes are not hidden by frequent classes. Weighted F1 is still reported because it reflects the dataset distribution, but it can be dominated by majority classes [11, 7]. The small gap between the two F1 scores suggests stable performance across both common and rare classes.

The training curves in Figure 3 show that the main models converged quickly after the first few epochs. Validation accuracy became very high and did not collapse later in training, so there was no clear evidence of overfitting in the selected models. The accuracy comparison shows that the best performance did not come from simply increasing model size.



(a) Training acc. **(b)** Validation acc.

Figure 3: Convergence across selected models.

The parameter and timing comparison shows that `Model 2cll` is useful because it reached 98.08% test accuracy with only 160,507 parameters, about 26.5% of `Model 2cl`. However, the runtime did not fall by the same ratio, because training time also depends on feature-map sizes, convolution operations, batch normalization, augmentation and optimizer behaviour. `Model 2cl RMS` kept the same parameter count as `Model 2cl`, but achieved better validation and test performance, so the improvement is explained by the training strategy rather than by extra computation.

Table 3 confirms the same pattern across the models set: the ResNet-style and Inception-style variants were useful exploratory comparisons, but they did not improve the final test result. The best model was therefore not the largest or most complex one, but the VGG-style architecture with the strongest training strategy.

Table 2: Main model comparison.

Model	Params	Test	Macro	Weighted
M2	1,721,035	97.30	0.9568	0.9725
M2cl	605,643	98.26	0.9670	0.9800
M2cll	160,507	98.08	0.97	0.98
M2cl RMS	605,643	**98.74**	**0.9794**	**0.9882**

Table 3: Accuracy comparison.

Model	Test	Train	Val.
M2	97.30	98.15	99.63
M2cl	98.26	99.19	99.90
M2cll	98.08	98.20	99.80
M2cl RMS	98.74	99.47	99.96
RMS ResNet	97.34	98.79	99.55
RMS Inception	97.24	97.24	99.55

6 Error Analysis and Limitations

The confusion matrices in Figure 4 are diagonal, especially for `Model 2cl RMS`, which confirms the high overall accuracy. The remaining errors are concentrated in visually similar classes. For the best model, the most frequent mistake was *Speed limit (120)* predicted as *Speed limit (80)* 12 times. Other repeated errors included *End speed limit(80)* and *Bumpy road*. The wrong-prediction tables in Tables 4 and 5 show both the most frequent confusion pairs and the classes that remain hardest to classify across models.

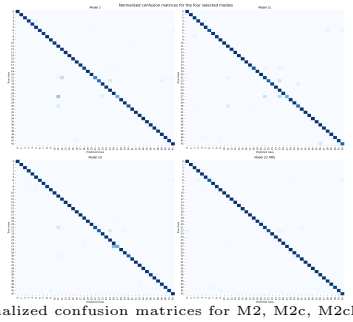


Figure 4: Normalized confusion matrices for M2, M2c, M2cl, and M2cl RMS.

Table 4: Final M2cl RMS errors.

Actual	Pred.	N
Speed 120	Speed 80	12
Ice/snow	Bicycles	9
End speed 80	EnP >3.5t	9
End speed 80	Speed 60	8
Bumpy road	Road work	6
Priority road	No veh.	6
Speed 80	Speed 60	5
Bumpy road	Speed 20	5
EnP >3.5t	End no pass	5
Gen. caution	Traffic sig.	4

Table 5: Hardest actual classes.

Class	M2	M2c	M2cl	RMS	Tot.
Gen. caution	34	57	15	6	112
Ice/snow	25	32	27	16	100
Bumpy road	29	26	24	14	93
Pedestrians	30	30	30	1	91
Speed 60	31	24	17	5	77
Speed 120	2	24	32	14	72
End speed 80	8	21	8	24	61
Double curve	31	7	13	7	58
Speed 70	20	8	10	4	42
Priority road	13	9	9	6	37

The class-level analysis Figure 5 shows that most classes have high precision, recall, and F1, but a few remain weaker. In `Model 2cl RMS`, *Pedestrians* improved compared with the earlier models, reaching an F1-score of 0.97. However, *Double curve* (F1 = 0.83), *Roundabout* (F1 = 0.89) and *End speed limit(80)* (F1 = 0.91) remained harder to classify. These errors are reasonable because some traffic signs share the same shape and differ only by small internal symbols.

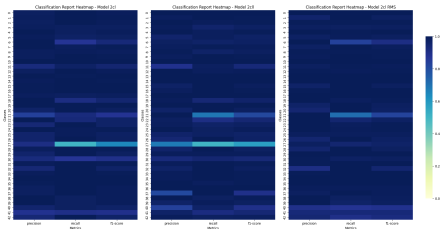


Figure 5: Classification report heatmaps for M2cl, M2cll, and M2cl RMS.

The main limitation is that this project classifies already cropped traffic signs. It does not detect signs in full road scenes, so it is not a complete autonomous-driving perception system. A real deployment would also need object detection, tracking and robustness to motion blur, weather variation, camera angle changes, occlusion, night-time images and country-specific sign distributions. Another limitation is the fixed 32×32 input size: it makes training efficient, but it can remove small details such as digits, arrows, or internal symbols, which probably contributed to errors between visually similar signs. Class imbalance also remains a challenge, especially for minority classes. Finally, although the nearest-neighbour distance check helped identify exact and near-duplicate risks, it was only a simple image-similarity sanity check rather than a full guarantee that the splits are completely independent. Since several final models were also compared on the test set, the reported accuracy should be read as the best result from these experiments, not as a single untouched test evaluation.

Future work could address these limitations by moving from cropped classification to full-scene traffic-sign detection, for example by combining a detector (YOLO) with the final CNN classifier. The model could also be tested at higher input resolutions to preserve small digits and symbols and with stronger augmentation for blur, rain, occlusion, night-time lighting and perspective changes. Finally, repeated stratified splits or cross-validation would make the model comparison more robust, while targeted augmentation or class weighting could improve performance on minority and visually similar classes.

Consequently, the experiments show that the best performance did not require the largest network. The strongest model was the VGG-style `Model 2cl RMS`, where normalization, data augmentation, batch normalization, dropout, global average pooling, RMSprop and learning-rate decay worked together to improve both generalization and training stability. This made it the most suitable final model, balancing predictive performance, computational cost and interpretability.

References

[1] Björn Barz and Joachim Denzler. Do we train on test data? purging cifar of near-duplicates. *Journal of Imaging*, 6(6):41, 2020. doi: 10.3390/jimaging6060041.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90. URL https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[4] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015. URL https://proceedings.mlr.press/v37/ioffe15.html.

[5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations (ICLR)*, 2014. URL https://arxiv.org/abs/1312.4400.

[7] David M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.

[8] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91. URL https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html.

[9] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(60), 2019. doi: 10.1186/s40537-019-0197-0.

[10] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. URL https://arxiv.org/abs/1409.1556.

[11] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45 (4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.

[12] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL https://jmlr.org/papers/v15/srivastava14a.html.

[13] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The german traffic sign recognition benchmark: A multi-class classification competition. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1453–1460, 2011. doi: 10.1109/IJCNN.2011.6033395. URL https://www.ini.rub.de/upload/file/1470692848_f03494010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf.

[14] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. doi: 10.1109/CVPR.2015.7298594. URL https://www.cv-foundation.org/openaccess/content_cvpr_2015/html/Szegedy_Going_Deeper_With_2015_CVPR_paper.html.

[15] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—rmsprop: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.